Data Platform Release Process

Proposal: testing ADF together with the Azure SQL config DB on the feature branch · Vamshi Singam · October 2026 · Follow-up to the release-gate demo

Summary

- **Problem:** config scripts for the Azure SQL configuration DB reach a database only after merging to `main`. ADF pipelines that need new config therefore can't be tested until after the merge, so untested work lands in `main` and fixes follow as extra PRs.
- **Proposal:** every feature branch gets its own short-lived config DB (a copy of the DEV config DB plus that branch's scripts). Developers debug ADF against it before raising the PR. The PR must carry the test run id.
- **Everything else stays as demoed:** config scripts travel inside the same PR as the ADF change, are cherry-picked by PR, built once, and deployed QA → UAT → PROD under the ServiceNow CR gate. Deploy order: config DB first, then Unity Catalog, then Databricks, then ADF.

1. What happens today



Today The DEV config DB is refreshed only when `main` changes, so the first real ADF test happens after the merge.

- A developer can debug ADF in the feature branch, but the config rows it reads aren't in any database yet.
- The merge to `main` becomes the test. A wrong config row breaks the shared DEV environment for everyone until a fix PR lands.
- Story + fix PRs then have to be picked together for QA, which makes cherry-picking harder.

2. Proposal: a config DB per feature branch



Proposed Each branch is tested against its own config DB before the PR is raised.

1. **Push** to `feature/AB#<story>-…` with ADF changes and config scripts together.

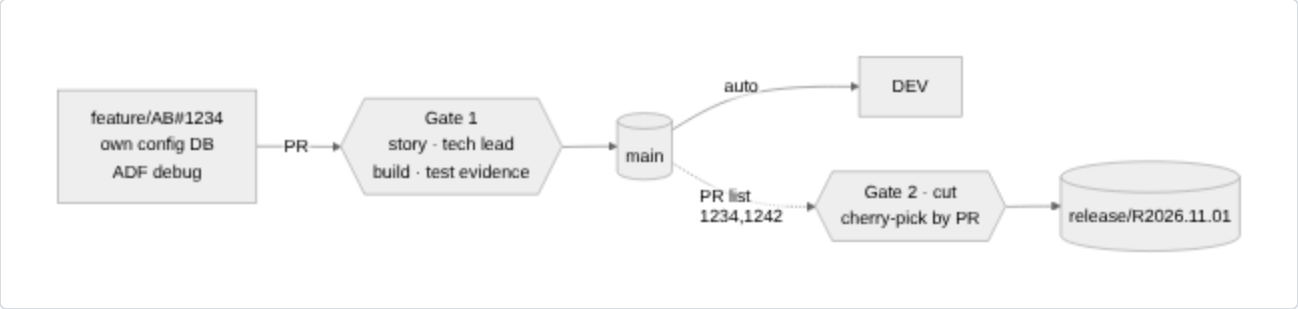
2. **Pipeline `feature-env`** copies the DEV config DB (which always equals `main`) to `sqldb-etlcfg-ft-<story>` in a small elastic pool and deploys the branch's schema (DACPAC) and config rows on top.
3. **Debug ADF in Git mode** with pipeline parameter `configDb = sqldb-etlcfg-ft-<story>`. The config linked service takes the database name as a parameter, so other developers and the shared DEV DB are untouched.
4. **Raise the PR** with `Feature-Test-Run: <ADF run id>` in the description. Gate 1 checks for it, alongside story link, tech-lead approval and the DACPAC build.
5. **After merge** DEV is refreshed as today (now an integration check, not the first test). A nightly job drops feature DBs whose branch is gone or that are older than 14 days.

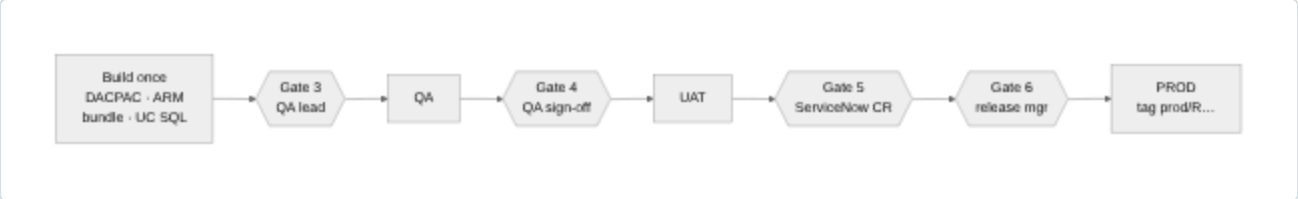
How the config DB is kept in source control

Kind	Example	How it is deployed
Schema	tables, procs, views	SQL project → DACPAC → `sqlpackage Publish` (drift-safe, blocks data loss)
Config data	`Data/sources/SAP_SALES.sql`	One idempotent `MERGE` file per config key in the post-deployment script; environment values (storage account, catalog) via SQLCMD variables
Runtime state	watermarks, run logs	Schema only. Rows are never deployed or overwritten

One file per config key matters: two PRs rarely touch the same file, so cherry-picking PRs into a release stays conflict-free.

3. Where this sits in the end-to-end release

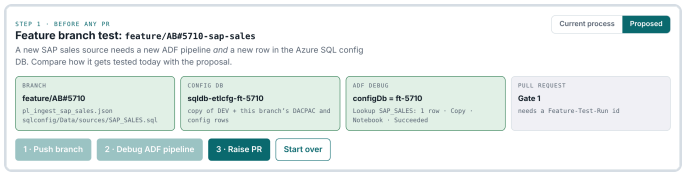




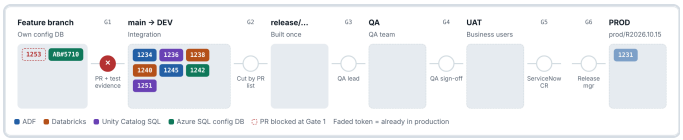
Feature test → PR → `main`/DEV → release cut by PR list → QA → UAT → ServiceNow CR → PROD.

Gate	Checks	Enforced by
0 · Feature test new	Branch tested against its own config DB; ADF debug run succeeded	`feature-env` pipeline
1 · PR → main	Story linked · tech lead approval · build green incl. DACPAC · Feature-Test-Run id · squash merge only	Azure Repos branch policies
2 · Release cut	Only the listed PRs; merged, linked, not already in prod; cherry-picked in merge order onto the last prod tag	Release pipeline
3 · QA	QA lead accepts the build	ADO environment approval
4 · UAT	QA sign-off recorded	ADO environment approval
5 · Change request	CR approved, in window, CR's PR list = release manifest	ServiceNow check
6 · PROD	Release manager go; deploy config DB → UC → Databricks → ADF; tag prod baseline; close CR and stories	ADO environment approval

4. Interactive simulator



Step 1: toggle *Current process* / *Proposed* and run push → debug → PR.



Then pick PRs and promote them through every gate.

To open it: save the attached `Release-Gate-Simulator.html` and open it in Edge or Chrome. It runs fully offline, with no install and no sign-in. It is also hosted at **https://vamshi455.github.io/enterprisedevops/** if your network allows GitHub.

5. Decisions needed from the team

1. **Copy of DEV vs. empty DB + seed.** Proposed: copy (realistic, includes everything already merged). An empty DB seeded from scripts is faster to create but needs full seed data in source control.
2. **Elastic pool sizing and cost owner** for feature DBs (suggest a small pool shared by all feature DBs, auto-dropped nightly).
3. **Tooling for config DB schema:** SQL project/DACPAC (proposed) or keep existing numbered scripts with a migration runner. Either works with this process.
4. **Branch naming** `feature/AB#<story>-…` so the pipeline can name the DB and link the story.
5. **Who reviews config data changes** in the PR (tech lead, or a config owner for shared sources).